[College logo placeholder]

Department of Computer Engineering

EXPERIMENT NO. 6

Case Study – Developer Automation System using MCP Protocol (Developer Tools Domain)

ReviewPilot: a pull-request review assistant over the Model Context Protocol

Course	Agentic AI Engineering and Enterprise Automation (AAIEA)
Name	[Student name]
Roll No.	[Roll number]
Experiment No.	6
Date	22-10-2026
Grade / Signature	

1. Aim, Objectives and Case Study

Aim: To design and implement a Developer Automation System using the Model Context Protocol (MCP) in which an AI agent communicates with developer tools through a standardised client-server architecture. The tasks covered are file listing and reading, code search, Python (AST) analysis, automated testing, tool discovery, sequential and parallel tool execution, validation, failure handling and recovery, execution tracing and performance monitoring.

Case study chosen: a pull-request review assistant (ReviewPilot). A developer opens a pull request (PR) and the system reviews it: it fetches the PR over MCP, plans which tools to run, runs the affected tests first and the whole suite only when they pass, turns tool results into validated findings, proposes a merge verdict (APPROVE / COMMENT / REQUEST_CHANGES / BLOCK) and lets a human decide what is posted and whether an automatic fix is applied. A second graph, the Developer Workbench, turns plain-language requests such as “Analyze calculator.py and run its tests” into multi-tool MCP plans, which reproduces the lab-manual scenario. The case fits “protocol-driven system integration and developer tooling” for three reasons: every capability (repository access, AST analysis, pytest, scanners, review posting) is a separate tool that must be discovered and called through one protocol; tools have very different risk (read-only, process-spawning, writing), which exercises the policy layer; and a review has real data dependencies (usages, then affected tests, then full suite), which exercises dependency-aware parallel scheduling.

1.1 Objectives

- Understand the architecture of the Model Context Protocol (host, client, server, tools, resources, JSON-RPC over stdio).
- Implement MCP servers in Python with FastMCP and an MCP client with ClientSession and the stdio transport.
- Discover tools dynamically (tools/list) and execute them (tools/call) with typed input and output schemas.
- Implement planner and supervisor agents, a safety and policy layer, and a validator.
- Demonstrate sequential and parallel execution and dependency-aware workflow execution; report DOP and DOS.
- Implement failure classification, controlled retry and recovery, execution tracing (TRACE and TASK ids, JSONL) and performance monitoring.
- Compare sequential and parallel execution of four independent tool calls over five runs (student Variation 9).

1.2 Tools and technologies

Table. 1: Tools and technologies used

Technology	Purpose
Python, asyncio	Core implementation; concurrent tool calls with asyncio.gather on a background event loop
MCP Python SDK 1.x (1.30.0 installed), FastMCP	Four MCP servers; tools registered with the @mcp.tool decorator, typed output schemas, annotations and _meta
ClientSession, stdio_client	One persistent stdio session per server; initialize, list_tools, call_tool, list_resources
LangGraph 1.x (1.2.14 installed)	StateGraph for the review and workbench workflows, parallel fan-out, interrupt() and Command(resume) for human approval, MemorySaver
jsonschema	Client-side validation of arguments (input schema) and results (output schema)
pytest, ast	Automated tests in an isolated checkout; AST analysis, complexity and symbol usage
Streamlit, Altair, pandas	Eleven-page interface, evaluation charts, MCP inspector
JSON, JSONL, logging	Reports, persistent trace records (mcp_traces.jsonl), execution.log

The manual lists Google Colab as the development environment; this implementation runs locally as a Streamlit application and a command-line tool and needs no API key (planning is rule based and deterministic).

2. Theory

2.1 Model Context Protocol

MCP is an open protocol that lets an AI application use external capabilities through one interface instead of a custom integration per tool. A **host** (here ReviewPilot) creates one **client** per **server**; each server exposes capabilities of three kinds: **tools** (functions the model or agent can invoke), **resources** (readable data addressed by URI) and **prompts** (reusable templates). Messages are JSON-RPC 2.0. In the stdio transport the client starts the server as a subprocess and exchanges messages over its standard input and output; this is how ReviewPilot launches its four servers (python -m reviewpilot.servers.<name>_server).

A session begins with **initialize**: client and server exchange protocol version, capabilities and identity, which is called capability negotiation. In this project the MCP Inspector page shows each server answering with protocol version 2025-11-25, serverInfo reviewpilot-repo, -analysis, -quality and -automation, and the capabilities experimental, prompts, resources and tools. The client then calls **tools/list**, which returns for each tool a name, description, a JSON Schema for the input (inputSchema), optionally a schema for the result (outputSchema) and **annotations**. A call is **tools/call** with the tool name and arguments; the result carries content (text) and, when the tool declares an output schema, **structuredContent**, a JSON object the client can validate and use without parsing text. A tool failure is returned as a result with isError set, which is different from a protocol error. Annotations (readOnlyHint, destructiveHint, idempotentHint, openWorldHint) are hints about behaviour; ReviewPilot reads them together with an explicit risk level stored in the tool metadata (_meta) when it decides what a tool may do. The servers define resources (repo://{repo}/readme, pr://{pr_id}/description, autofix://rules) but no prompt templates.

2.2 MCP compared with plain function calling and REST

Plain function calling binds tool definitions to one application and one model API: the definitions live in the prompt-building code and the functions run in the same process, so adding a tool means changing the agent. With MCP the tools live in separate servers and are **discovered at run time**; the Supervisor in this project calls tools/list before every review and rejects any task whose tool is not offered or whose arguments do not match the schema. A REST API offers similar remote access but has no common discovery, schema, annotation or session convention, so every client needs bespoke code for each service. MCP standardises exactly those parts, at the cost of a session (process start, about 0.7 s per server here) and one more layer to secure; the measured effect of keeping sessions open is shown in Section 5.3.

2.3 Multi-agent orchestration with LangGraph

The workflow is a LangGraph **StateGraph**: each agent is a node that reads and writes a shared typed state (ReviewState), and edges define the order. After the Executor the graph **fans out** to four reviewer nodes that run in the same super-step, and a join edge waits for all four before the Validator. **Human approval** uses interrupt(): the graph pauses and returns a payload (verdict, findings, fixable rules); the application resumes it later with Command(resume=decision) using the MemorySaver checkpointer, so nothing is posted or written until a person decides. A conditional edge after approval routes to the FixAgent only when a person approved at least one auto-fix rule.

2.4 Tool risk levels and safety

Every tool has a risk level: **LOW** (read-only), **MEDIUM** (starts a process or posts a review) and **HIGH** (writes files or pushes a commit). The PolicyGuard allows LOW and MEDIUM tools, checks every path argument with safe_project_path() (which rejects absolute paths, drive letters, NUL bytes and any path that resolves outside the repository), rejects test targets with shell metacharacters, and requires a human for HIGH. The automatic review never calls a HIGH tool. Text inside a pull request (title, description, commit messages, added comments) is treated as untrusted data: the IntakeGuard scans it for instructions aimed at the reviewer (override, forced verdict, role-play) and the SecurityReviewer reports them as finding SEC003; the text is never used as an instruction.

2.5 Failure classification and controlled retry

Errors carry a category prefix and the client classifies each result before deciding what to do. Only transient categories are retried, with a limit and exponential back-off; retrying an input or security error cannot succeed and only wastes calls. This is the manual's Table 4 as implemented.

Table. 2: Failure classification and recovery action (as implemented)

Category	Example	Action in ReviewPilot
INPUT	File not found, argument of the wrong type, unknown tool	Stop, no retry. Schema errors are rejected by the client before any round trip (attempts = 0)
TOOL	Tool returned an error result	Stop and report; the review lists the evidence gap
SECURITY	Path outside the project, unsafe test target	Stop, no retry
TIMEOUT	pytest or a tool exceeds its timeout	Retry, at most 2 times, back-off 0.05 s doubled each attempt
EXECUTION	Unexpected failure, transport failure, server process crash	Retry (limited); after a crash the client reconnects (new stdio session) and repeats the call

2.6 Degree of parallelism (DOP) and degree of sequential execution (DOS)

DOP is the number of independent tool calls that can run at the same time; DOS is the amount of work that must stay sequential because of dependencies. In this project the Executor groups tasks into phases by their dependencies: calls inside a phase run with asyncio.gather (bounded by a semaphore, max_parallel = 8) and phases run one after another. It reports DOP as the widest phase and DOS as the number of phases; in sequential mode every call is its own step (DOP 1, DOS = number of calls). Parallelism helps when calls take long or wait; for very short calls, scheduling and the single stdio connection per server dominate, which the Variation 9 measurements show.

3. System Design

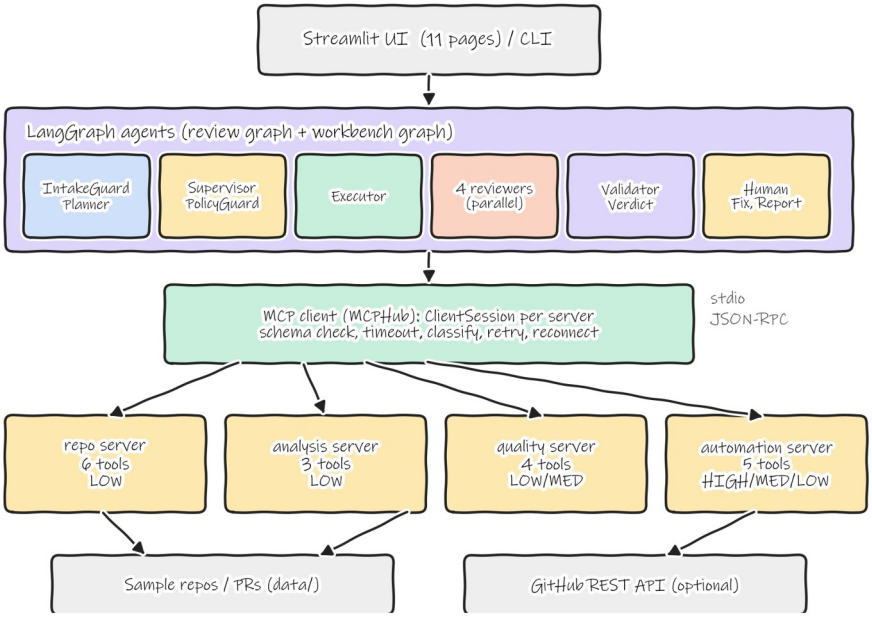


Fig. 1: Architecture of ReviewPilot: UI, LangGraph agents, MCP client, four MCP servers and the repositories behind them

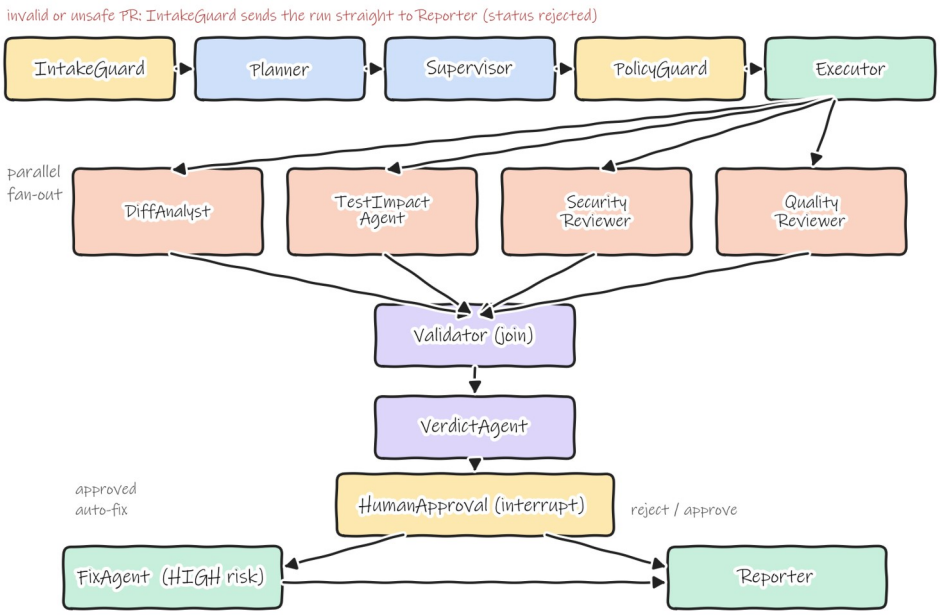


Fig. 2: Review workflow (LangGraph): guard, plan, supervise, check policy, execute, four parallel reviewers, validate, verdict, human approval, optional fix, report

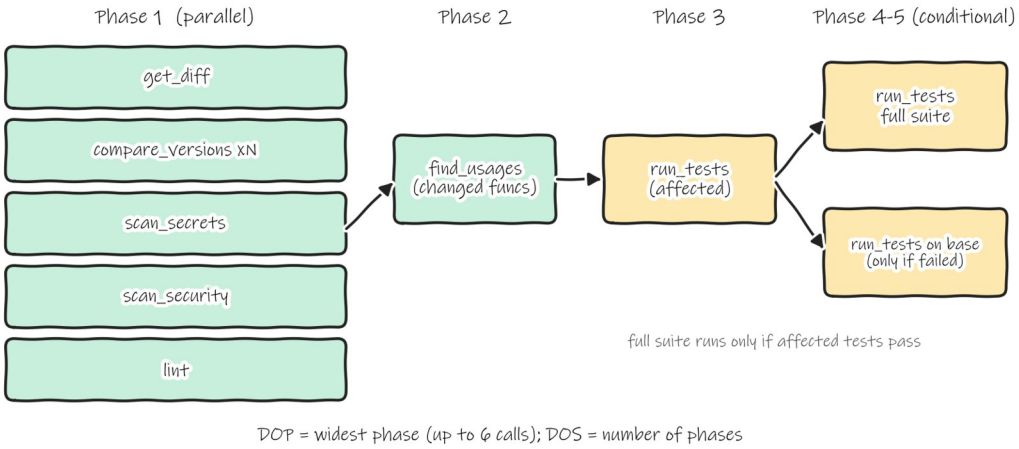


Fig. 3: Executor phases for a typical PR; independent calls share a phase and run with asyncio.gather

3.1 MCP tools

The four servers expose 18 tools (the manual asks for five: list_files, read_file, search_code, analyze_python, run_tests, all of which are present). Risk is declared by the server in each tool's _meta and annotations and read by the client at discovery.

Table. 3: MCP tools, servers, purpose and risk level

Tool	Server	Purpose	Risk
list_files	repo	List files of a repository or of a PR head	LOW
read_file	repo	Read a file or line range (path-confined)	LOW
search_code	repo	Text or regex search over source files	LOW
list_pull_requests	repo	List open pull requests	LOW
get_pull_request	repo	PR metadata and changed files	LOW
get_diff	repo	Added and removed lines of every changed file	LOW
analyze_python	analysis	AST analysis: functions, complexity, imports, TODOs	LOW
compare_versions	analysis	Main vs PR: functions, complexity delta, unused imports, tests	LOW
find_usages	analysis	Where a symbol is used; whether tests use it	LOW
run_tests	quality	pytest in an isolated checkout with timeout	MEDIUM
lint	quality	Debug calls, bare except, mutable defaults, complexity	LOW
scan_secrets	quality	Hard-coded credentials (values masked)	LOW
scan_security	quality	eval/exec, shell calls, unsafe deserialisation, SQL by string	LOW
create_autofix_branch	automation	Apply safe fixes on a review branch	HIGH
push_autofix_commit	automation	Push the fix as one commit to the PR branch	HIGH
delete_autofix_branch	automation	Revert a fix branch	MEDIUM
post_review	automation	Post verdict and inline comments (GitHub or stored)	MEDIUM
get_review	automation	Read the stored review of a PR	LOW

4. Implementation

4.1 Agents

Fourteen agents form the review graph; the workbench graph reuses the Supervisor, PolicyGuard and Executor and adds RequestGuard, TaskPlanner, ResultValidator and Summarizer. A reviewer owns its rules, so a finding can be traced to the agent and the tool result that produced it.

Table. 4: Agents of the review workflow

Agent	Role	Input	Output
IntakeGuard	Validates the PR id, fetches the PR, screens its text, checks paths	PR id	pr, guard (injection hits)
Planner	Changed files to tool tasks with dependencies	pr	plan (depends_on, resolver, condition)
Supervisor	Fresh tools/list; each task must name an offered tool and match its schema	plan	approved plan, checks
PolicyGuard	Risk level and path/target checks per task	plan, tool risks	allow / deny decisions
Executor	Runs the plan over MCP in phases, parallel within a phase	plan, denied set	results, timeline, DOP/DOS metrics
DiffAnalyst	Files, size bucket, DIF001 for very large PRs	get_diff result	diff summary, findings
TestImpactAgent	TST001 failing tests (base re-run separates pre-existing), TST002 removed tests, TST003 untested functions	run_tests, find_usages	test report, findings
SecurityReviewer	SEC001 secrets, SEC002 dangerous calls, SEC004 SQL, SEC003 reviewer-directed text	scan results, guard	findings
QualityReviewer	QLT001 to QLT007 on added lines	lint, compare_versions	findings
Validator	Tool status and output schema; finding must cite a known rule, a changed file and an added line	results, findings	validated findings, dropped, gaps
VerdictAgent	Severity policy to verdict, score, confidence; never approves without test evidence	validated findings	verdict
HumanApproval	interrupt(): approve, reject, override, dismiss, approve auto-fix rules	verdict, findings	human decision, final findings
FixAgent	Fix branch, re-run tests, push or roll back (HIGH risk)	approved rules	fix result
Reporter	post_review and developer_automation_report.json	everything above	report, posted review

4.2 Important source code

Excerpts are copied from the repository; the file name precedes each excerpt.

reviewpilot/servers/repo_server.py – FastMCP server and a LOW-risk tool

```
mcp = FastMCP("reviewpilot-repo", instructions="Read-only access to GitHub
repositories, the bundled sample repositories and their pull requests.",
log_level="WARNING")
   ...
@mcp.tool(annotations=READ_ONLY, meta=risk_meta("LOW"))
async def list_files(repo: str, pr_id: str = "", pattern: str = "*", ref: str = "head") ->
FileList:
   """List the files of a repository (at the pull-request head when pr_id is given).
`pattern` is a glob such as *.py."""
   def work():
      files = _files(repo, pr_id, ref)
      sel = [{"path": p, "lines": len(c.splitlines()), "size": len(c.encode())} for p, c in
sorted(files.items()) if fnmatch.fnmatch(p, pattern) or fnmatch.fnmatch(p.split("/")
[-1], pattern)]
      return {"repo": repo, "ref": ref if pr_id else "base", "count": len(sel), "files":
sel}
   return await run_blocking("list_files", work)
```

reviewpilot/servers/common.py – annotations and tagged errors

```
READ_ONLY = ToolAnnotations(readOnlyHint=True, destructiveHint=False,
idempotentHint=True, openWorldHint=False)
RUNS_PROCESS = ToolAnnotations(readOnlyHint=False, destructiveHint=False,
idempotentHint=True, openWorldHint=False)
WRITES = ToolAnnotations(readOnlyHint=False, destructiveHint=True,
idempotentHint=False, openWorldHint=False)
   ...
def tagged(category: str, message: str) -> ToolError:
   """Errors carry a [CATEGORY] prefix so the client can classify them (input /
security / timeout / tool / execution)."""
   return ToolError(f"[{category}] {message}")
```

reviewpilot/servers/automation_server.py – a HIGH-risk tool

```
@mcp.tool(annotations=WRITES, meta=risk_meta("HIGH"))
async def create_autofix_branch(pr_id: str, rules: list[str]) -> FixBranch:
```

```
    """Apply safe mechanical fixes (rules QLT001, QLT002, QLT004, SEC001) to the pull request on a separate review branch."""
    return await run_blocking("create_autofix_branch", create_fix_branch, pr_id, list(rules))
```

reviewpilot/mcp_client.py – classification and schema validation

```
RETRYABLE = {"TIMEOUT", "EXECUTION"}
CATEGORY_ACTION = {"INPUT": "stop (no retry)", "TOOL": "stop / report", "SECURITY": "stop (no retry)", "TIMEOUT": "retry (limited)", "EXECUTION": "retry (limited)"}
    ...
def classify(message: str) -> str:
    m = message or ""
    for tag in ("SECURITY", "INPUT", "TIMEOUT", "TOOL", "EXECUTION"):
        if f"[{tag}]" in m:
            return tag
    low = m.lower()
    if "unknown tool" in low or "validation error" in low or "input should be" in low or "missing required" in low or "field required" in low:
        return "INPUT"
    if "timed out" in low or "timeout" in low:
        return "TIMEOUT"
    return "EXECUTION"
    ...
    def validate_arguments(self, tool: str, arguments: dict) -> str | None:
        info = self.tools().get(tool)
        if info is None:
            return f"Unknown tool: {tool}"
        try:
            jsonschema.validate(arguments, info.input_schema)
        except jsonschema.ValidationError as exc:
            where = "/".join(str(p) for p in exc.absolute_path) or "arguments"
            return f"invalid argument {where}: {exc.message}"
        return None
```

reviewpilot/mcp_client.py – call path: validate, call, classify, retry

```
        if s.validate_arguments:
            problem = self.validate_arguments(tool, args)
            if problem:
                trace.record(task_id=task_id, tool=tool, server=server, arguments=args, status="rejected", duration_ms=0, attempt=0, error=problem,
                             category="INPUT", phase=phase, agent=agent, risk=risk)
                return ToolResult(tool, server, False, error=problem, category="INPUT", task_id=task_id, attempts=0)
    ...
        while attempt < max_attempts:
            attempt += 1
            t0 = time.perf_counter()
            started = t0 * 1000
            seen = self.conns[server].connects
            try:
                data, err = await self._once(tool, args, timeout, fresh=not s.reuse_connections)
                cat = classify(err) if err else None
    ...
            dur = (time.perf_counter() - t0) * 1000
            ok = err is None
            trace.record(task_id=task_id, tool=tool, server=server, arguments=args, status="success" if ok else "failed", duration_ms=dur,
                         attempt=attempt, error=err, category=cat, phase=phase, agent=agent, risk=risk, started_ms=started)
            result = ToolResult(tool, server, ok, data=data, error=err, category=cat, duration_ms=round(dur, 2), attempts=attempt, task_id=task_id,
                                output_valid=self.validate_output(info, data) if ok else None)
            if ok:
                break
            retryable = s.retry_policy == "naive" or cat in RETRYABLE
            if not retryable or attempt >= max_attempts:
                break
            if s.retry_policy == "controlled":
                await asyncio.sleep(s.retry_backoff_s * 2 ** (attempt - 1))
        return result
```

reviewpilot/planning.py – dependency-aware scheduling

```
    level: dict[str, int] = {}
    remaining = {t["id"]: t for t in tasks}
    for _ in range(len(tasks) + 1):
        progressed = False
        for tid, t in list(remaining.items()):
            deps = [d for d in t["depends_on"] if d in {x["id"] for x in tasks}]
            if all(d in level for d in deps):
                level[tid] = 1 + max((level[d] for d in deps), default=-1)
                del remaining[tid]
                progressed = True
        if not progressed:
            break
    if remaining:
        raise ValueError("cyclic task dependencies: " + ", ".join(remaining))
    phases: list[list[dict]] = []
    for t in tasks:
        lv = level[t["id"]]
        while len(phases) <= lv:
            phases.append([])
        phases[lv].append(t)
    return phases
```

reviewpilot/executor.py – condition and phase execution

```
def condition_holds(task: dict, results: dict, settings: Settings) -> tuple[bool, str]:
    c = task.get("condition")
    if not c or not settings.dependency_aware:
        return True, ""
    if c == "affected_tests_passed":
        dep = results.get(task["depends_on"][0]) if task["depends_on"] else None
        if dep is None or dep.get("status") == "skipped":
            return True, "no affected tests were selected, so the whole suite runs"
        if dep.get("ok") and dep["data"]["status"] in ("passed", "no_tests"):
            return True, "affected tests passed"
        return False, "affected tests failed - the full suite would only repeat the failure"
    if c == "head_tests_failed":
        bad = [r for r in results.values() if r["tool"] == "run_tests" and r.get("ok") and r["data"]["status"] in ("failed", "error")]
        return (True, "tests failed on the PR") if bad else (False, "all tests passed - no baseline needed")
    return True, ""


    ...
        if settings.execution_mode == "sequential":
            outs = []
            for t, a in calls:
                outs.append(await one(t, a, i))
        else:
            outs = await asyncio.gather(*(one(t, a, i) for t, a in calls))
```

reviewpilot/agents/review.py – Supervisor

```
    hub = hub_of(state)
    listed = {t["name"]: t for t in hub.discover()}
    checks, approved = [], []
    for t in state["plan"]:
        if t["tool"] not in listed:
            checks.append({"task": t["id"], "tool": t["tool"], "result": "rejected", "detail": "tool not offered by any connected MCP server"})
            continue
        if t.get("resolver"):
            checks.append({"task": t["id"], "tool": t["tool"], "result": "deferred", "detail": f"arguments resolved at run time ({t['resolver']}); validated before the call"})
            approved.append(t)
            continue
        problem = hub.validate_arguments(t["tool"], t["args"])
        if problem:
            checks.append({"task": t["id"], "tool": t["tool"], "result": "rejected", "detail": problem})
            continue
        checks.append({"task": t["id"], "tool": t["tool"], "result": "valid", "detail": f"{listed[t['tool']]['server']} server, schema ok"})
        approved.append(t)
    rejected = [c for c in checks if c["result"] == "rejected"]
    return {"plan": approved, "supervisor": {"checks": checks, "tools_listed": len(listed), "rejected": len(rejected)},
            "_summary": f"{len(listed)} tools listed; {len(approved)} tasks approved, {len(rejected)} rejected"}
```

reviewpilot/agents/review.py – PolicyGuard decision

```
def policy_decide(task: dict, risk: str | None, settings, repo: str | None, approved_high: bool = False) -> tuple[str, str]:
    if risk is None:
        return "deny", "unknown tool"
    if settings.enforce_policy:
        args = task.get("args", {})
        paths = [args[k] for k in PATH_ARGS if isinstance(args.get(k), str)] + [p for p in args.get("paths", []) if isinstance(p, str)]
        if repo:
            for p in paths:
                try:
                    safe_project_path(repo_root(repo), p)
                except Exception as exc:  # noqa: BLE001
                    return "deny", f"path security: {exc}"
        for tgt in args.get("targets", []) if isinstance(args.get("targets"), list) else []:
            if ".." in tgt or not re.match(r"^[A-Za-z0-9_./:\[\]-]+$", tgt):
                return "deny", f"unsafe test target '{tgt}'"
    order = ["LOW", "MEDIUM", "HIGH"]
    if order.index(risk) >= order.index(settings.require_approval_for) and settings.enforce_policy:
        if approved_high:
            return "allow", f"{risk} risk - approved by a human"
```

```
        return "needs_approval", f"{risk} risk - requires human approval"
    if risk == "MEDIUM":
        return "allow", "MEDIUM risk - allowed with timeout protection and an isolated checkout"
    return "allow", "LOW risk - read-only"
```

reviewpilot/agents/review.py – Validator (finding checks)

```
    kept, dropped, seen = [], [], set()
    for f in state.get("findings", []):
        reason = None
        if f["rule"] not in RULES:
            reason = "unknown rule"
        elif f["path"] not in changed and not (f["rule"] == "SEC003" and f["path"] == "PR description") and f["rule"] != "TST001":
            reason = "cites a file outside the pull request"
        elif f["rule"] not in EXEMPT_LINE_RULES and f["line"] not in added.get(f["path"], set()):
            reason = "cites a line the pull request did not add"
        key = (f["rule"], f["path"], f["line"])
        if reason is None and key in seen:
            reason = "duplicate"
        if reason:
            dropped.append(f | {"dropped_because": reason})
            continue
        seen.add(key)
        kept.append(f)
```

reviewpilot/graph.py – graph wiring

```
    g.add_edge(START, "intake_guard")
    g.add_conditional_edges("intake_guard", R.route_after_intake, {"planner": "planner", "reporter": "reporter"})
    g.add_edge("planner", "supervisor")
    g.add_edge("supervisor", "policy_guard")
    g.add_edge("policy_guard", "executor")
    for n in REVIEWERS:
        g.add_edge("executor", n)
    g.add_edge(REVIEWERS, "validator")
    g.add_edge("validator", "verdict")
    g.add_edge("verdict", "human_approval")
    g.add_conditional_edges("human_approval", R.route_after_human, {"fix_agent": "fix_agent", "reporter": "reporter"})
    g.add_edge("fix_agent", "reporter")
    g.add_edge("reporter", END)
```

5. Observations and Results

Data source and reconstruction notice. The benchmark folder of the repository (reviewpilot/benchmark, git-ignored) was missing, so it was rebuilt for this report: three sample projects (calculator from the lab manual, inventory, textkit) with passing tests, and 62 labelled pull requests (15 calculator, 24 inventory, 23 textkit) whose labels were written by hand from the intent of each change (reviewpilot/benchmark/repos.py, catalog.py, build.py; python -m reviewpilot.benchmark.build). Every figure below comes from running the system on this reconstruction: python -m pytest (34 tests, all passed), python -m reviewpilot.cli eval (storage/reports/run_20261022_125905, 248 s) and the running application. Numbers obtained with another benchmark would differ.

5.1 Tool-call trace (mcp_traces.jsonl)

One complete review: PR CALC-01 (“Add power() with tests”), clean, final verdict APPROVE, run from the command line with an automatic approval. The executor ran 9 calls in 4 phases (DOP 6, DOS 4) and skipped one task (no base re-run was needed); the Reporter then called post_review. Each call is one JSON record in the trace file.

Table. 5: Tool-call trace of TRACE-C2F9EF2C (review of CALC-01)

Trace ID	Task ID	Tool	Server	Phase	Risk	Status	Time (ms)
TRACE-C2F9EF2C	TASK-001	get_pull_request	repo	0	LOW	success	14.1
TRACE-C2F9EF2C	TASK-002	get_diff	repo	1	LOW	success	23.7
TRACE-C2F9EF2C	TASK-003	compare_versions	analysis	1	LOW	success	33.7
TRACE-C2F9EF2C	TASK-005	scan_secrets	quality	1	LOW	success	44.0
TRACE-C2F9EF2C	TASK-004	compare_versions	analysis	1	LOW	success	53.7
TRACE-C2F9EF2C	TASK-006	scan_security	quality	1	LOW	success	59.3
TRACE-C2F9EF2C	TASK-007	lint	quality	1	LOW	success	62.5
TRACE-C2F9EF2C	TASK-008	find_usages	analysis	2	LOW	success	8.4
TRACE-C2F9EF2C	TASK-009	run_tests	quality	3	MEDIUM	success	1123.6
TRACE-C2F9EF2C	TASK-010	run_tests	quality	4	MEDIUM	success	1192.0
TRACE-C2F9EF2C	TASK-011	post_review	automation	99	MEDIUM	success	9.0

Tasks 002 to 007 share phase 1 and start within 7 ms of each other (39.9 to 46.1 ms after the first call), while find_usages, the affected tests and the full suite start only after their dependencies have finished. The two run_tests calls (1.12 s and 1.19 s) dominate the 2.41 s wall time, so the busy time (2.60 s) exceeds the wall time only by a factor of 1.08. post_review (phase 99) ran last. The files written by the system are listed below.

Table. 6: Artifacts generated in storage/mcp_logs/ and storage/reports/

File	Purpose
execution.log	One line per agent step and per tool call (trace id, task id, status, time)
mcp_traces.jsonl	One JSON record per tool call: trace_id, task_id, tool, server, arguments, status, execution_time_ms, timestamp, error, error_category, attempt, phase, agent, risk
developer_automation_report.json, runs/TRACE-*.json	Full report of the last review and of every persisted review (plan, supervisor checks, policy, execution, findings, human decision)
server_repo.log, server_analysis.log, server_quality.log, server_automation.log	stderr of each MCP server process
reports/run_*/ (report.json, one CSV per suite), latest_report.json	Evaluation output

The manual lists experiment_06_complete_report.json and experiment_06_execution_summary.json. These two files are not produced by this implementation; their content is covered by developer_automation_report.json and reports/latest_report.json.

5.2 Results per test suite

Table. 7: Results of the five evaluation suites

Suite	Size	Result
Review benchmark (labelled PRs, offline samples)	62 PRs	verdict accuracy 95.2%; finding precision 100.0%, recall 95.5%, F1 97.7% (TP 63, FP 0, FN 3); false alarms on clean PRs 0.0%; BLOCK recall 100.0%; unsafe merges 0.0%; failing tests caught 100.0%; tool results valid 100.0%; average 1.856 s (p95 2.538 s), 8.4 tool calls, average DOP 5.06
Developer Workbench (natural-language requests)	32 requests	plan accuracy 93.8%; tool precision 100.0%, recall 94.6%, F1 97.2%; outcome accuracy 93.8%; 0.206 s per request
Security tests	26 tests	25 passed, 1 failed (96.2%)
Failure-injection scenarios	16 scenarios	16 passed (100.0%)
Human-in-the-loop scenarios	12 scenarios	12 passed (100.0%)
Automated tests (pytest, tests/)	34 tests	34 passed (unit, MCP protocol, workflows, GitHub end-to-end against a fake GitHub API)
Quality index (weighted mean of the above)	-	97.1

Verdict confusion matrix of the benchmark (label to ReviewPilot verdict): APPROVE 23 correct and 2 given COMMENT; COMMENT 5 correct and 1 given REQUEST_CHANGES; REQUEST_CHANGES 26 of 26; BLOCK 5 of 5. Accuracy by category is 100% for every category except prompt injection (75%) and untested behaviour change (0 of 2); see Section 7.

Table. 8: Failure-injection scenarios (FL-01 to FL-16, all passed): behaviour per failure type

Failure type	Scenario and observed behaviour
Execution error	Transient lint failure: retried once, succeeds (attempts 2). Persistent failure: stops after 3 attempts. Server process crash mid-call and during post_review: client reconnects and retries (attempts 2)
Timeout	Tool hangs past its timeout: TIMEOUT, retried, second attempt succeeds. Diff tool keeps timing out: limited retries, review completes with verdict COMMENT and the gap listed
Tool / dependency error	Tool error: stop and report, attempts 1. Scan unavailable: review completes and reports the gap. Tests cannot run on a clean PR: APPROVE is downgraded to COMMENT. All compare_versions fail: find_usages skipped, full suite still runs
Input / security error	Missing file, missing argument (attempts 0), path outside project, invalid pytest timeout: stopped with INPUT or SECURITY, no retry. Naive policy retries a security error 5 times, which shows why classification matters

5.3 Optimisation techniques

Table. 9: Optimisation techniques of the manual (nine), measured with the basic and the optimised approach

Technique	Metric	Basic: value	Optimised: value	Change
Parallel execution	Variation 9 wall time (ms)	await each call: 73.5	asyncio.gather per phase: 59.1	19.6%
Parallel execution	4 independent pytest runs (ms)	await each call: 4613.8	asyncio.gather: 1652.3	64.2%
Parallel execution	review workflow executor time (ms)	sequential executor: 1964	phased parallel executor: 1902.9	3.1%
Dependency-aware scheduling	pytest runs on 3 failing PRs	linear chain, full suite always: 9	phases + skip full suite when affected tests fail: 6	33.3%
Dependency-aware scheduling	avg executor time on failing PRs (ms)	linear chain: 3567.1	phased DAG: 2288.4	35.8%
Tool-level validation	server round trips for 5 malformed calls	send and let the server fail: 5	JSON-schema check against tools/list first: 0	100.0%
Controlled retry	calls spent on 4 failure scenarios	retry every error 5x: 17	retry only TIMEOUT / EXECUTION, max 2: 5	70.6%
Timeout protection	time to a result when a tool hangs 3 s (ms)	wait for the tool: 3015	1 s timeout + retry: 1074.2	64.4%
Path security	traversal attempts stopped before reaching a server (of 6)	server-side check only: 0	PolicyGuard + safe_project_path: 6	-
Structured logging	required trace fields per call (of 8)	free-text log line: 0	JSON record per call: 8	-
Persistent tracing	records of one review readable after a restart	in-memory list: 0	append-only mcp_traces.jsonl: 10	-
Performance monitoring	metrics captured per workflow	none: 0	wall, busy, DOP, DOS, speed-up, retries, skipped, failed: 8	-
Connection reuse (extra)	4 calls (ms)	new stdio session per call: 3510.2	persistent sessions: 98.8	97.2%

Reading the table: parallel execution and dependency-aware scheduling reduce wall time and avoid work (the full suite is skipped when the affected tests already fail: 9 to 6 pytest runs on three failing PRs); client-side schema validation removes five server round trips for five malformed calls; controlled retry spends 5 calls instead of 17 on four failure scenarios; a 1 s timeout returns in about 1.1 s from a tool that hangs for 3 s; path security, structured logging, persistent tracing and monitoring are measured as counts of what is captured or stopped (for example all six traversal attempts stopped before reaching a server). Persistent sessions, an extra technique beyond the manual, cut four calls from 3510 ms to 99 ms.

5.4 Student variation 9: performance comparison (sequential vs parallel)

The same four independent tool calls (list_files, read_file, search_code, analyze_python on the calculator project) were run through the MCP client sequentially and in parallel; the experiment was repeated five times. The code is run_performance and VAR9_CALLS in evaluation/suites.py.

Table. 10: Sequential vs parallel execution time (4 tool calls, 5 runs)

Run	Sequential time (ms)	Parallel time (ms)
1	76.8	67.5
2	72.5	52.6
3	68.2	56.3

Run	Sequential time (ms)	Parallel time (ms)
4	81.6	60.5
5	68.6	58.8
Mean	73.5	59.1

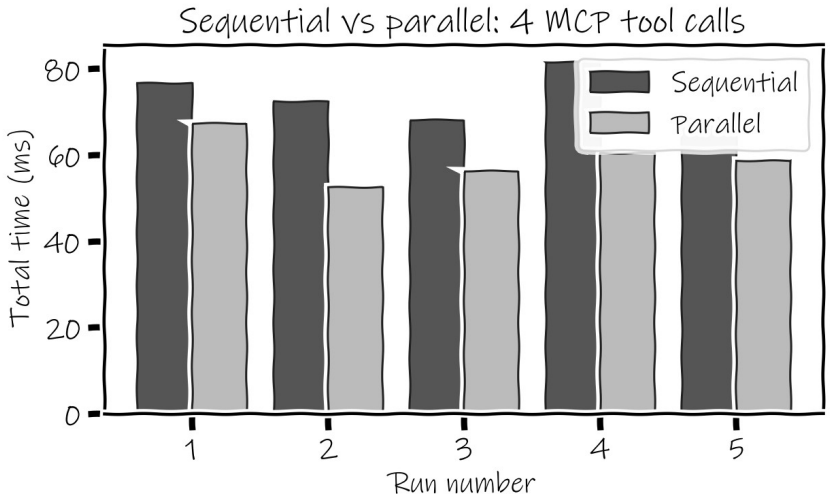


Fig. 4: Comparison of sequential and parallel execution time over five runs

Table. 11: Performance comparison summary

Metric	Value
Number of tasks	4
Number of runs	5
Average sequential time	73.5 ms
Average parallel time	59.1 ms (standard deviation 5.0 ms vs 5.1 ms sequential)
Average time per tool (sequential)	18.4 ms
Speed-up (sequential / parallel)	1.24x
Time reduction	19.6 %

Observation: parallel execution was faster in all five runs and by about 20 % on average (1.24x), but the absolute saving is only about 14 ms, and the spread of the five runs is similar for both modes. The calls take about 18 ms each, so scheduling, the thread hand-off inside the server and a single stdio connection per server are comparable to the work itself. When the tool time is large the benefit is clear: four independent pytest runs took 4613.8 ms sequentially and 1652.3 ms in parallel (2.79x), while the review workflow, where most of the time is the two pytest calls that depend on earlier phases, improved by only 1.03x.

5.5 DOP and DOS

Table. 12: DOP and DOS of the measured workloads

Workload	DOP	DOS	Execution type
Variation 9, parallel (4 independent calls)	4	1 phase	Fully parallel
Variation 9, sequential	1	4 steps	Sequential
Review phase 1 (get_diff, compare_versions per file, scan_secrets, scan_security, lint)	5 to 6 (sampled PRs)	-	Parallel
Review phases 2 to 4 (find_usages, affected tests, full suite, base re-run)	1 each	2 to 3 phases	Sequential, dependency-aware
Review workflow overall (8 sampled PRs, parallel mode)	average 5.5	average 3.62 phases	Hybrid (parallel + sequential)

Table. 13: Review workflow, sequential vs parallel executor (8 sampled PRs)

PR	Calls	DOP	DOS	Sequential (ms)	Parallel (ms)	Speed-up
CALC-01	9	6	4	2357	2246	1.05x
CALC-03	7	5	3	1152	1119	1.03x
INV-01	9	6	4	2367	2226	1.06x
INV-09	8	6	3	1237	1155	1.07x
INV-14	8	5	4	2385	2288	1.04x
TXT-01	9	6	4	2246	2251	1.00x
TXT-04	7	5	3	1184	1138	1.04x
TXT-07	8	5	4	2783	2801	0.99x

A higher DOP only pays off when the parallel calls are long enough and the machine has the capacity; the semaphore (max_parallel = 8) bounds server load, and in this workload the longest tasks (pytest) sit in later, dependent phases, so the overall gain stays small.

6. Screenshots of the Running Application

Taken with Playwright against streamlit run Home.py on the offline sample repositories (the rebuilt benchmark). The pull request shown is CALC-02, which removes the zero check from divide().

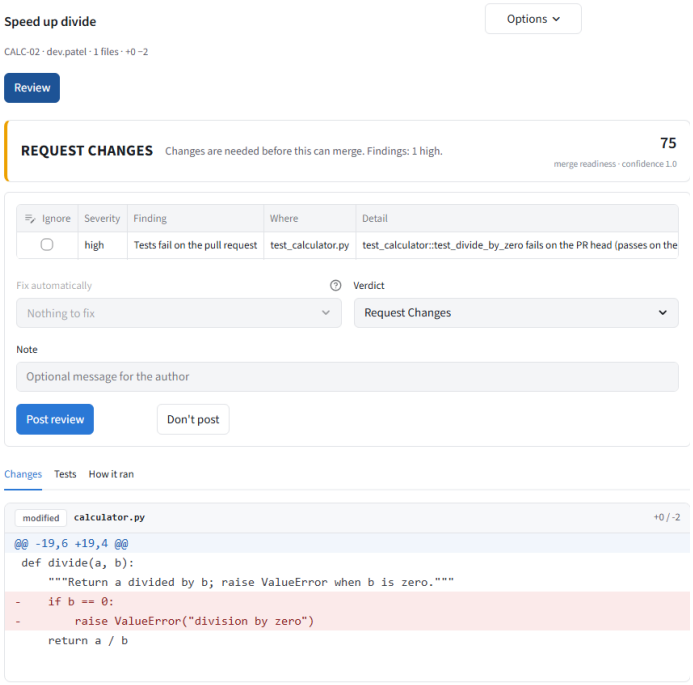


Fig. 5: Pull requests page after a review: verdict banner, finding and the Changes tab

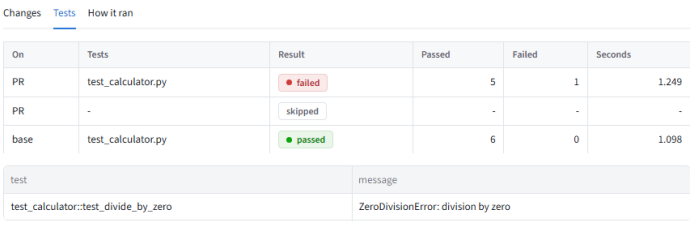


Fig. 6: Tests tab: affected tests fail on the PR, the same tests pass on the base, full suite skipped

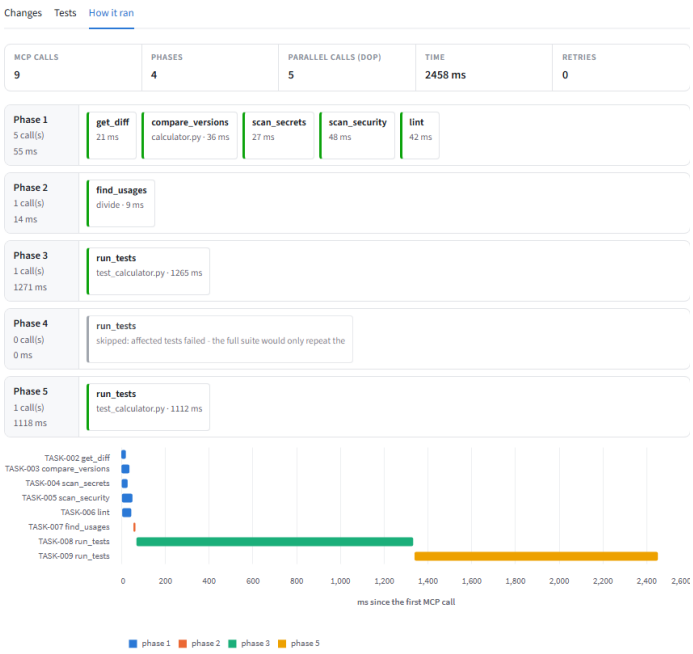


Fig. 7: How it ran tab: phases, parallel calls (DOP 5) and timeline of the MCP calls

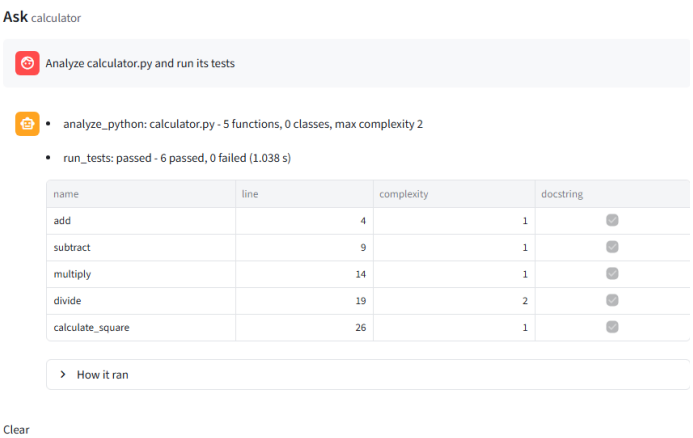


Fig. 8: Ask page (Developer Workbench): “Analyze calculator.py and run its tests”

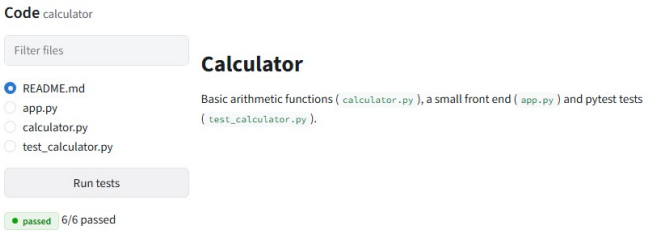


Fig. 9: Code page: files of the sample project and its test run

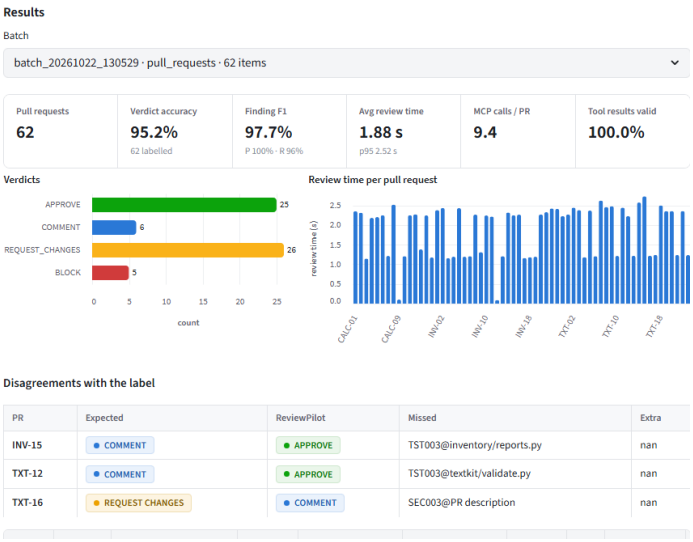


Fig. 10: Batch testing: the 62-PR benchmark, verdicts and disagreements with the label



Fig. 11: Overview page (How it works): MCP servers with their tools, the review stages and the review graph

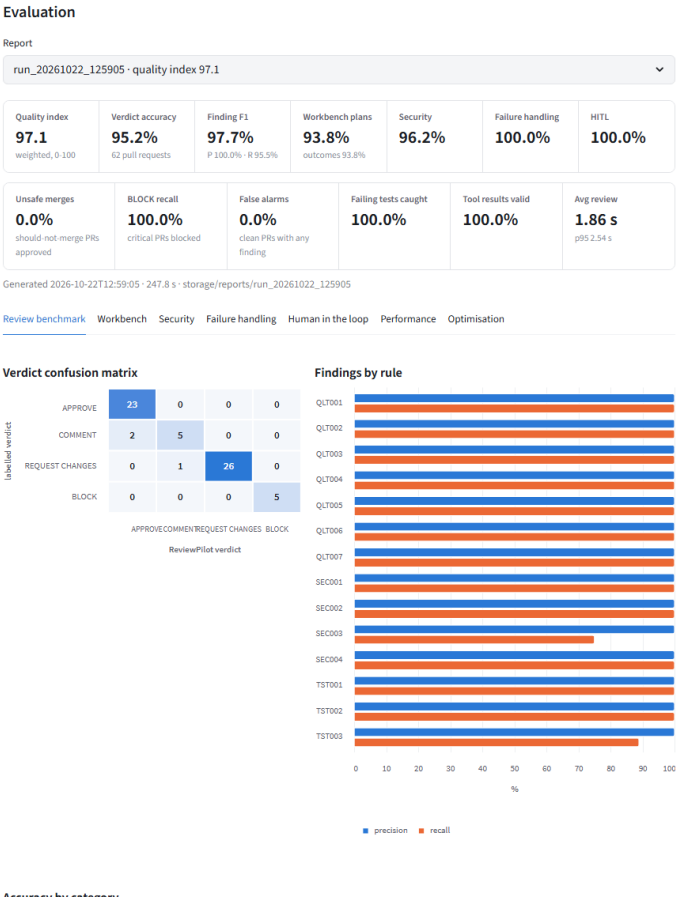


Fig. 12: Evaluation report: scorecard, verdict confusion matrix and findings by rule

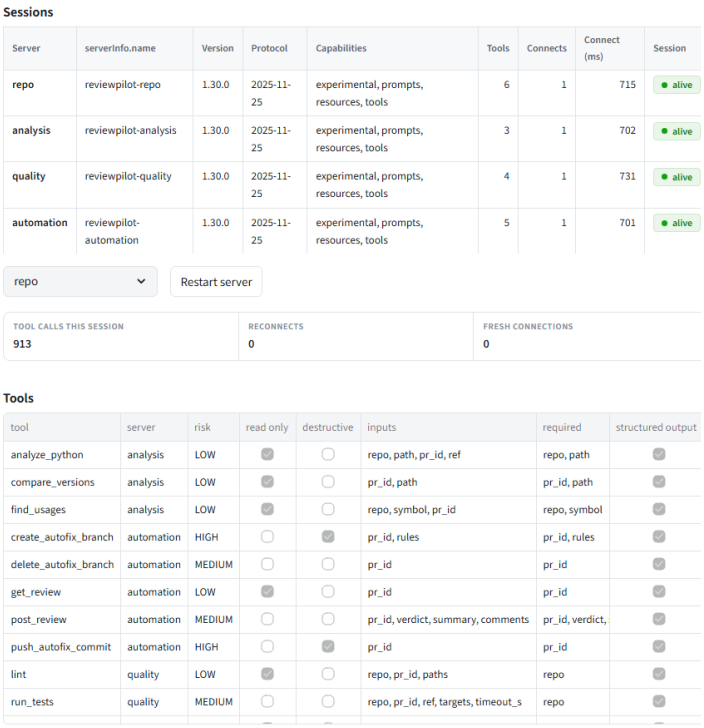


Fig. 13: MCP inspector: sessions (initialize result per server) and tools/list with risk, hints and schemas

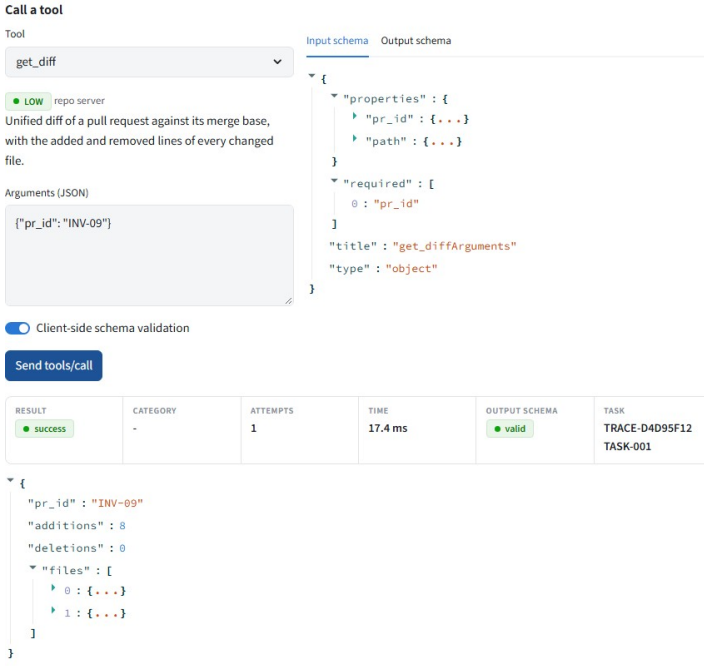


Fig. 14: MCP inspector: tools/call get_diff for INV-09, structured result, output schema valid, TRACE and TASK id

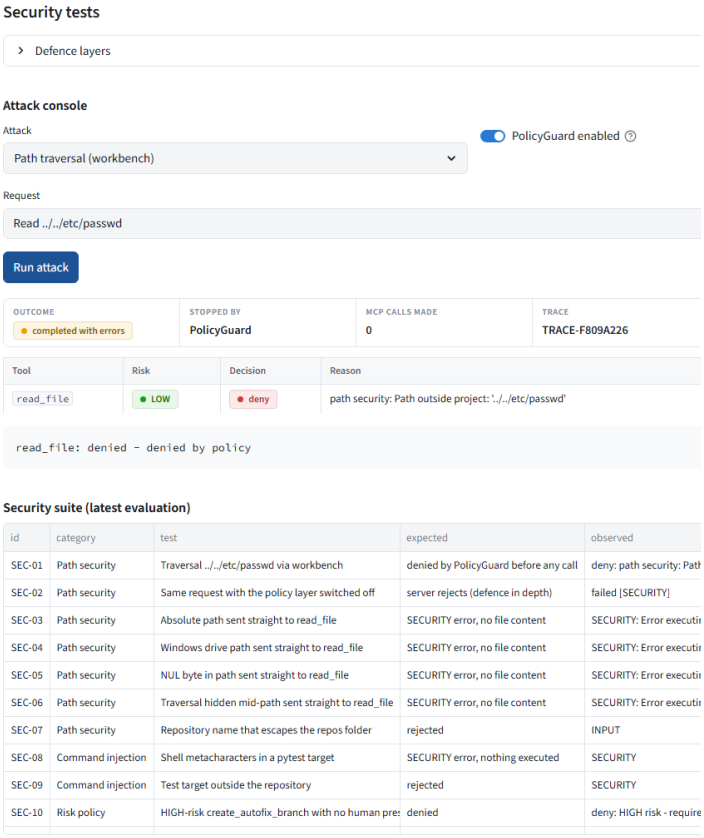


Fig. 15: Security tests: a path traversal request is denied by the PolicyGuard before any MCP call

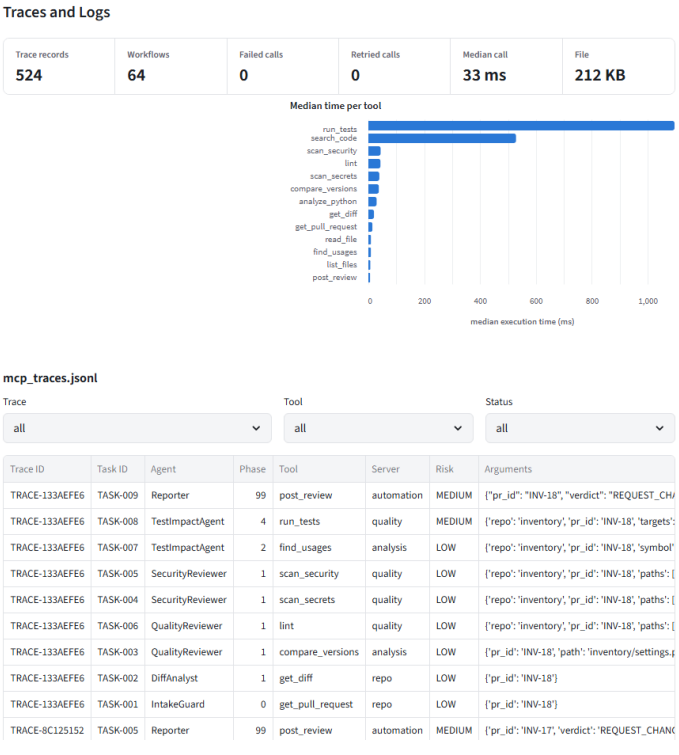


Fig. 16: Traces and logs: mcp_traces.jsonl records and median time per tool

7. Discussion

What worked. All four MCP servers connect over stdio, negotiate capabilities and expose 18 tools whose input and output schemas are used by the client; 100% of tool results in the benchmark were valid against their output schema. The review benchmark gave no false positive (precision 100%), no false alarm on clean PRs, caught every PR that must be blocked (5 of 5) and every PR with failing tests, and approved no PR that should not merge. Dependency-aware scheduling avoided running the full suite after failing affected tests, controlled retry recovered from transient failures and a crashed server (16 of 16 failure scenarios), and the human gate behaved as designed in all 12 scenarios (pause, approve, reject, override, dismiss, approved fix, rollback, and no fix without a person).

Known limits. The report lists these itself. (1) INV-15 and TXT-12 are untested behaviour changes to functions that already have tests; the review approves them (expected COMMENT) because TST003 only checks that a changed function is referenced by some test, not whether the tests cover the new behaviour. (2) TXT-16 is a pull request whose description politely asks the reviewer to approve without any trigger phrase; SEC003 is not raised and the verdict is COMMENT instead of REQUEST_CHANGES. (3) In the workbench, two phrasings without a file name or the word “usages” are not understood (WB-28 “What does the pricing module do?”, WB-32 “Is divide covered by tests?”), which is why plan accuracy is 93.8%. The planner is rule based, so it asks the user to rephrase rather than guess.

The failing security test. Of 26 security tests, 25 pass and SEC-23 fails (96.2%): a paraphrased request to approve, with none of the trigger words, is not flagged as SEC003 and the verdict is COMMENT. Detection of reviewer-directed text is pattern based, so a sufficiently rephrased sentence passes the screen. The damage is contained, not removed: PR text is never executed as an instruction, no write tool runs in an automatic review, and approving or posting needs a human in manual mode, but the injection attempt is not reported to the reviewer. The other 25 tests (path traversal at the policy layer and, with the policy switched off, at the server; shell metacharacters in test targets; HIGH-risk tools without a human; destructive and shell requests; unknown tools and wrong argument types; role-play and override injection; secret masking) pass.

Performance. Parallel execution of very short tool calls gave a small gain (1.24x, about 14 ms), in line with the manual’s own observation that parallelism helps little for short tasks; the larger gains come from running independent long tools in parallel (2.79x for four pytest runs), from skipping work through dependency-aware scheduling (one third fewer pytest runs on failing PRs) and from keeping MCP sessions open (about 35x for four calls). **Implementation notes.** The manual’s five tools are a subset of the 18 implemented; the README states 17 tools in two places while the servers register 18, and the Inspector lists 18. Two manual artifact files (experiment_06_*) are not generated, as stated in Section 5.1.

8. Conclusion

The experiment implemented a developer automation system on the Model Context Protocol: four FastMCP servers with 18 typed, risk-labelled tools, a ClientSession-based client that discovers tools, validates arguments and results, classifies failures and retries only transient ones, and a LangGraph workflow of fourteen agents with a planner, supervisor, policy guard, validator and human approval. On the evaluation of the rebuilt benchmark it reached a quality index of 97.1, with no unsafe merge, 100% on the failure-handling and human-in-the-loop scenarios and 96.2% on the security tests, and the measurements show that parallel execution helps most for long-running tools and little for very short ones. The known gaps, paraphrased instructions and untested behaviour changes, are left visible instead of being tuned away.

References

- Model Context Protocol, specification (lifecycle, transports, tools, resources, prompts), https://modelcontextprotocol.io/specification
- Model Context Protocol Python SDK (FastMCP, ClientSession, stdio_client), https://github.com/modelcontextprotocol/python-sdk
- LangGraph documentation (StateGraph, parallel branches, interrupts and human-in-the-loop), https://langchain-ai.github.io/langgraph/
- Lab Manual, Experiment 6: Developer Automation System using MCP Protocol (Developer Tools Domain), VESIT.

Inferences